

Prime-Encoded Tensor Calculus (PETC v2)

This document formalizes a PETC “v2” that is strong enough to (i) type-check tensor products and contractions, (ii) track **axis provenance** through reshape/split/merge, and (iii) support **proof-carrying contraction plans**. It includes a concrete **reshape witness algebra** designed to be lightweight in Python while mapping cleanly into Lean.

1. Core algebra

1.1 Generator set

Let G be a set of **axis atoms** (semantic generators). In practice:

- Python: fresh symbols/integers (often called “primes” but they need not be literal primes).
- Lean: an arbitrary type G .

1.2 Signatures

Define the **signature group** as the free abelian group on G :

$$\text{Sig} := \mathbb{Z}^{(G)} = \{e : G \rightarrow \mathbb{Z} \text{ with finite support}\}.$$

- Addition: $(e + f)(g) = e(g) + f(g)$ - Zero: $0(g) = 0$ - Negation: $(-e)(g) = -e(g)$

For reshape/provenance we also use the **positive cone**:

$$\text{Sig}^+ := \mathbb{N}^{(G)} \subset \text{Sig}.$$

(axes carry nonnegative multiplicities of atoms).

1.3 Multiplicity (optional numeric realization)

A “numeric realization” is a monoid homomorphism

$$\mu : \text{Sig}^+ \rightarrow \mathbb{N}_{>0}$$

interpreting atoms as integer factors. For example, if atoms are mapped to distinct primes, μ becomes a product of primes to powers.

Important: PETC v2 treats $e \in \text{Sig}^+$ as the primary object; $\mu(e)$ is optional runtime evidence.

2. Typed tensors

2.1 Variance

Let $\Sigma := \{+1, -1\}$ (covariant/contravariant sign).

2.2 Axes and shapes

An **axis** is a pair

$$\text{Axis} := (\sigma, e) \in \Sigma \times \text{Sig}^+.$$

A **shape** is a finite list of axes:

$$\text{Shape} := \text{List}(\text{Axis}).$$

2.3 Global signature invariant

Given a shape $S = [(\sigma_i, e_i)]_{i=1}^n$, define its **global signature**

$$\text{sig}(S) := \sum_{i=1}^n \sigma_i \cdot e_i \in \text{Sig}$$

(where $+1 \cdot e = e$, $-1 \cdot e = -e$).

A tensor value T carries a shape S , and PETC's core invariant is that operations preserve (or transform predictably) $\text{sig}(S)$.

3. Certified operations (typing rules)

Below, write S for shapes and $S ++ S'$ for list concatenation.

3.1 Tensor product

If $T : S$ and $U : S'$, then

$$T \otimes U : S ++ S'.$$

Invariant: $\text{sig}(S ++ S') = \text{sig}(S) + \text{sig}(S')$.

3.2 Contraction (pairwise)

A contraction removes one covariant and one contravariant axis with the **same positive signature**.

If S has axes $(+1, e)$ and $(-1, e)$ at positions i / j , then contracting those axes yields $\text{contract}(S; i, j)$ obtained by deleting both.

Legality condition: same $e \in \text{Sig}^+$ and opposite variance.

Invariant: $\text{sig}(\text{contract}(S; i, j)) = \text{sig}(S)$.

3.3 Permutation

Any axis permutation π yields $\text{perm}(S, \pi)$.

Invariant: $\text{sig}(\text{perm}(S, \pi)) = \text{sig}(S)$.

3.4 Reshape / split / merge (provenance-aware)

Reshape is not “free”; it is witnessed by a **proof object** that the axis atoms are merely regrouped.

A reshape acts only on axes with the **same variance sign**; it refactors the list of positive signatures.

4. Reshape witness algebra

4.1 Motivation

Standard runtime shape checks only guarantee integer sizes match. PETC v2 aims to also guarantee that reshapes do not mix semantic atoms incorrectly.

We achieve this by requiring a **witness** that the reshape is built from primitive steps that preserve the multiset of atoms.

4.2 Positive-axis factorization view

Every axis signature $e \in \text{Sig}^+$ is a finite multiset of atoms. Splitting and merging axes correspond to splitting/merging multisets:

$$\text{split: } e \rightsquigarrow e_1, e_2 \quad \text{where } e = e_1 + e_2.$$

[\text{text}